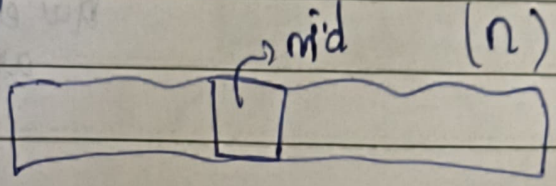
Strassen

w. this DNC is done

T.C. of all the DNC algos covered:

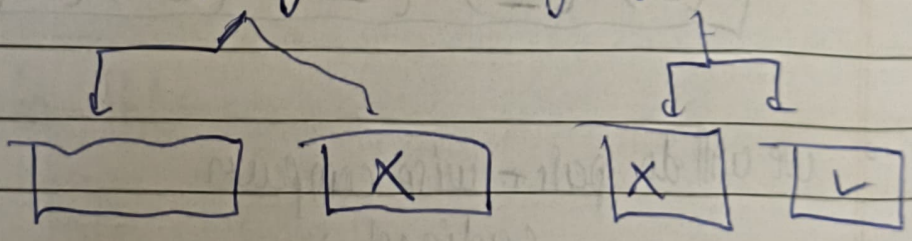
(i) Binary Search



if key == mid return

if key < mid

if key > mid



$(\frac{n}{2})$ $(\frac{n}{2})$ $(\frac{n}{2})$ $(\frac{n}{2})$

$$T(n) = T\left(\frac{n}{2}\right) + 1$$

using master theorem

$a = 1$ $k = 0$

$b = 2$ $p = 0$

$\log_b a = 0$

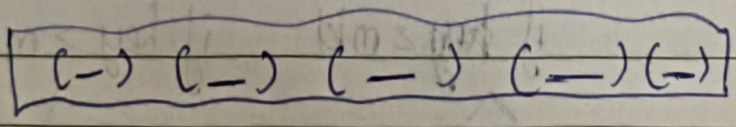
$\therefore \Theta\left(n^{\log_b a} \cdot (\log n)^{p+1}\right)$

$\Rightarrow \Theta(\log n)$

(ii) maximum and minimum in DNC of an array

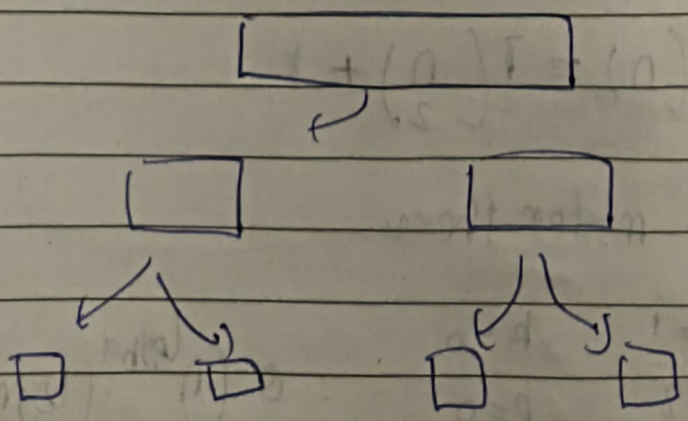
traditional method → go thru array & save
O(2n)
where 2n → no. of comparisions btw elements of arr

in this method of DNC



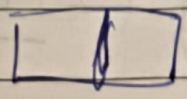
- we will do pair-wise comparison adjacent ;
- and we'll split to halves

i.e.



if ~~bottom size~~ ☐ size = 1, then return the element

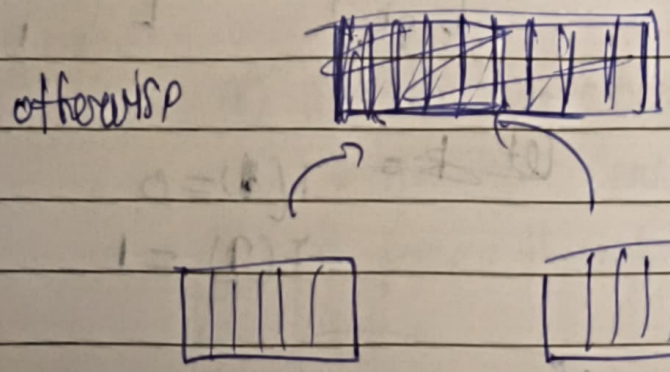
itself as max & min also its very important that you add base case



(2 elements)

* it is that base case which reduces T.C.

here u only need 1 comparisions to find min & max,



in this condition, you'll need two comparisons

so, if $T(1) = 0$

$T(2) = 1$

where $T(n)$ represents no. of comparisions operations in elements

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + 2$$

$$\Rightarrow 4 \cdot T\left(\frac{n}{4}\right) + 2 \cdot 2 + 2$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + \underbrace{2+4+8+\cdots}_{k \text{ terms}}$$

use GP

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + 2\left[\frac{2^k - 1}{1}\right]$$

~~let k=~~ $T(1) = 0$

$T(2) = 1$

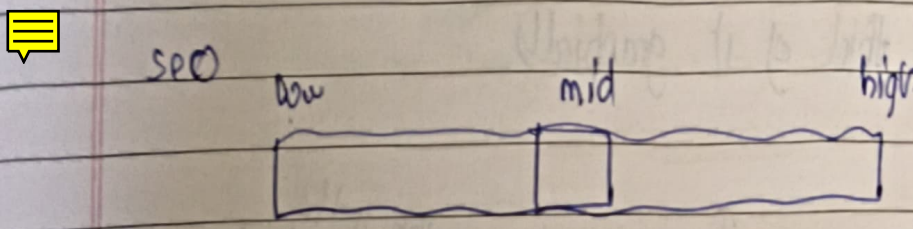
$$\therefore \boxed{\frac{n}{2^k} = 2}$$

$$\therefore T(n) = \frac{n}{2} \cdot T(2) + 2\left[\frac{n}{2} - 1\right]$$

$$T(n) = \frac{n}{2} + \frac{2n}{2} - 1$$

$$\boxed{T(n) = \frac{3n}{2} - 1}$$

(iii) Maximum sum subarray: ↳ DNC approch is nlog n



if low == high
return arr[low]

3 cases:

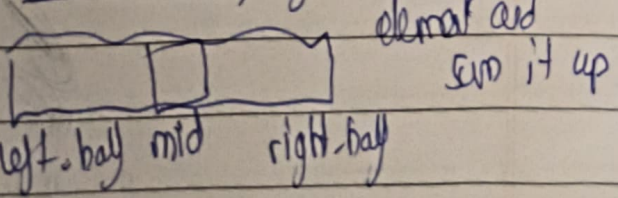
① Maximum sum subarray is to our left (low · mid)

② it passes through mid [has to find · mid]

③ " " to our right (low · mid)

$$T(n) = \underbrace{T\left(\frac{n}{2}\right)}_{\text{left}} + \underbrace{T\left(\frac{n}{2}\right)}_{\text{right}} + \underbrace{f(n)}_{\text{for middle}}$$

for middle



- keep doing sum += arr[i]
  l_max = max(l_max, arr[i]);

same for right

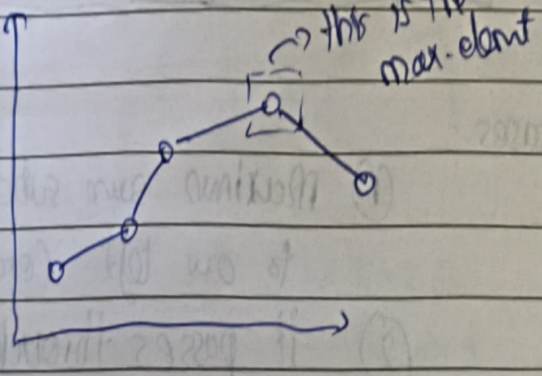
~~then return l_max + r_max;~~

$O(\log_2 n)$

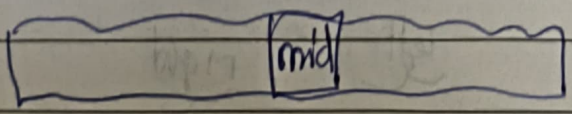
(iv) Maximum element in Unimodal array

↳ increasing then decrease

- think of it graphically

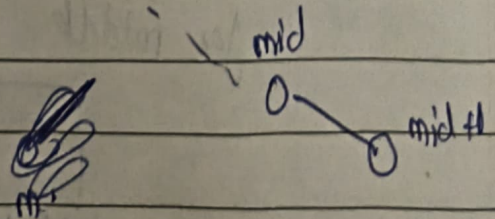


now



if (arr[mid] > arr[mid+1])

i.e.



∴ the max is to the left

search left

search right

mid mid+1

mid+1

Illy

if (arr[mid] <= arr[mid+1])

{

if (low == high)

{return arr[low];

}

$\therefore T(n) = 1T\left(\frac{n}{2}\right) + 1$

$a = 1, \quad k = 0$

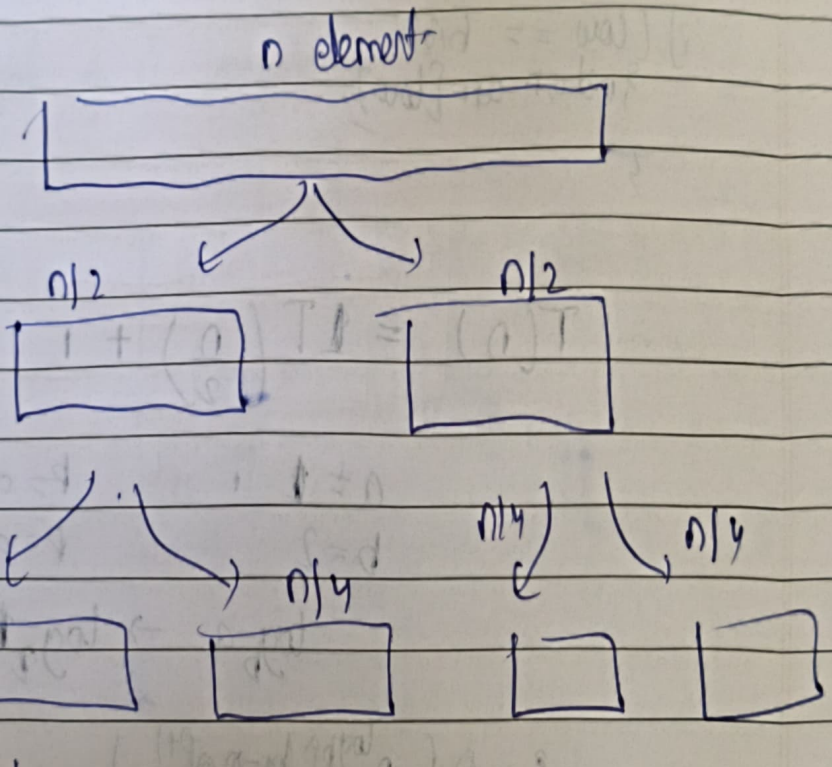
$b = 2 \quad p = 0$

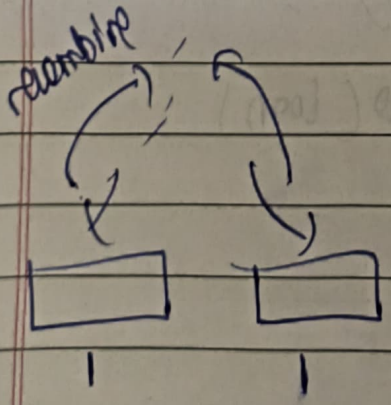
$\log_b a \rightarrow \log_2 1 \rightarrow 0$

$\therefore \Theta(n^{\log_b a} (\log n)^{p+1})$

$\Rightarrow \Theta(\log n)$

(v) Merge Sort algo $O(n \log_2 n)$





* you will get already sorted halves

* so for recombining you just have to run a 2 pointer approach

that's it -

left half right half

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n$$

n : time to combine two halves of $\frac{n}{2}$

$$\therefore T(n) = 2T\left(\frac{n}{2}\right) + n$$

$a = 2$, $b = 2$, $\log_b a = \log_2 2 = 1$, $k = 1$, $p = 0$

$$\therefore \theta(n^{\log_b a} (\log n)^{p+1})$$

$$\Rightarrow \theta(n \log n)$$

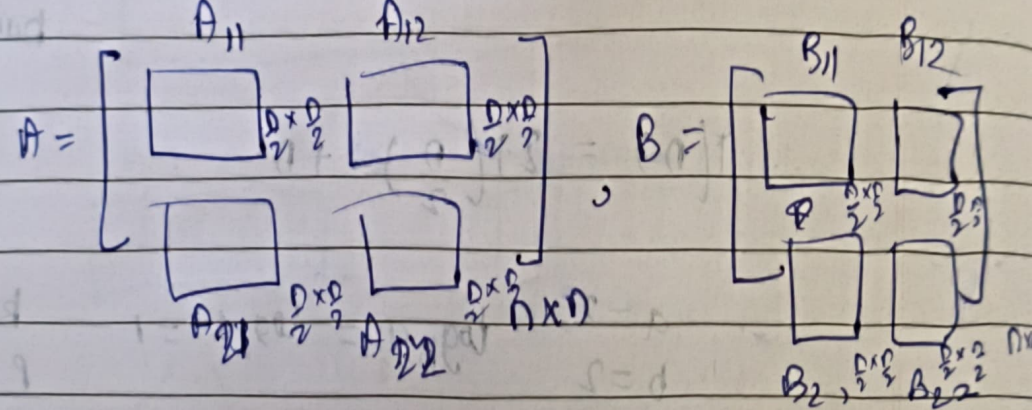
(vi) Strassen's multiplication algorithm

$$A = \begin{bmatrix} \cdots \\ \cdots \end{bmatrix}_{n}, \quad B = \begin{bmatrix} \cdots \\ \cdots \end{bmatrix}_{n}$$

Let $T(n)$ be Total no. of operations for multiplying two matrices of $n \times n$ size

∴ for A×B

T(n) = ?



for ordinary multiplication,

$$T(n) = 8T\left(\frac{n}{2}\right) + \Theta(n^2)$$

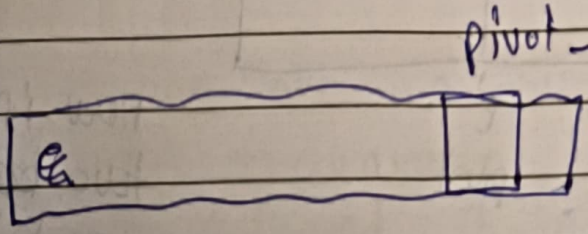
here we'll be recursively calling only 7 multiplication operations on each of the $\frac{n}{2}$ matrices

$$\therefore T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$$

$a = 7$, $b = 2$, $\log_b a = 2.81$, $k = 2$, $p = 0$

$\therefore T.C. \rightarrow \Theta(n^{\log_b a}) \Rightarrow \Theta(n^{2.81})$

(vii) Quick Sort



°so what well do first is based on our pivot we will partition to two parts,

whoever is <= pivot_ele well put in left

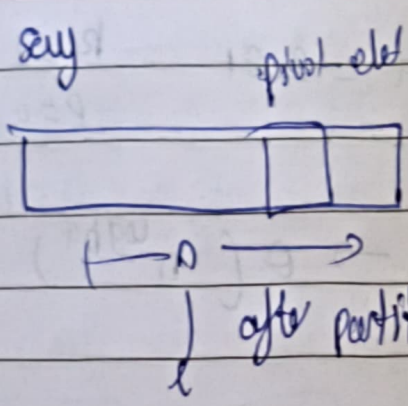
whoever is >= pivot_ele well put in right

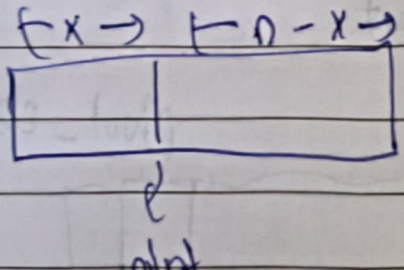
°after partition, pivot is in right Pate

$\Theta(n)$

now well split into two parts

Let's say





T(n) = no. of operations to sort n elements

now in these two ~~partitions~~ parts, well recursively call the quicksort function

$$\therefore T(n) = T(\alpha) + T(n-\alpha) + \Theta(n)$$

- best case: ~~and~~.

- best case occurs when pivot divides array to 2 equal halves

$$\therefore T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + \Theta(n)$$

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n)$$

on solving using master theorem

$$\Theta(n^{\log_b a} (\log n)^{p+1}) \rightarrow \Theta(n \log n)$$

- average case :

if we use a reasonably random pivot, on an average, we will get pretty balanced halves,

$$\therefore \Theta(n \log n)$$

- worst case :

if our pivots are very bad, and are only separating 1 (i.e. pivot is smallest or largest)

$$\therefore T(n) = T(1) + T(n-1) + \Theta(n)$$

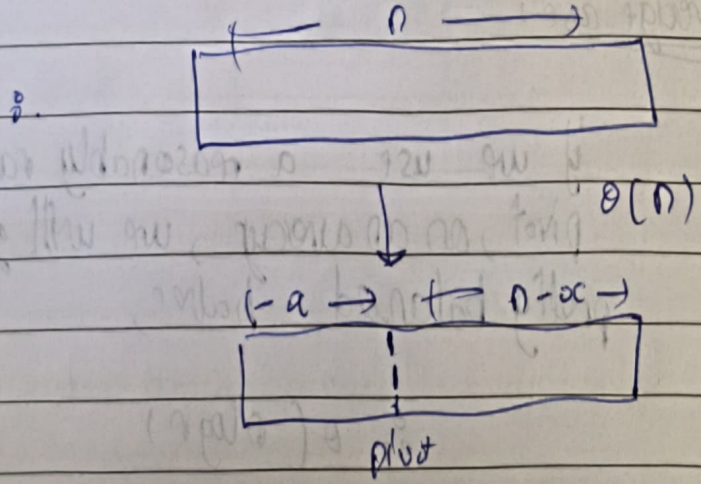
$T(1)$ ↙ no. of operations to sort 1 element is zero

$$\therefore T(n) = T(n-1) + \Theta(n)$$

T.C. → $\Theta(n^2)$

(viii) Median of an array using quick select $O(n)$

- well use the same partitioning idea as quick sort



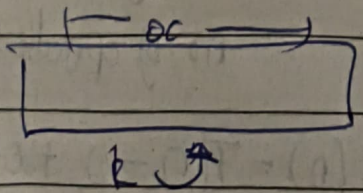
now if we want kth smallest

(for median $\frac{n}{2}$ th smallest)

what we'll do is:

see all behind pivot is smaller than pivot.

now if $a > k$



length x can also be obtained by

classmate
Date
Page

∴ kth smallest is in left partition,

otherwise it's in the right part,

so @ every step, we'll keep discarding a partition.

∴ $T(n) = T(\text{correct partition}) + \Theta(n)$

(a) Best case

if all the time we're splitting it into two halves

∴ $T(n) = T(\frac{n}{2}) + \Theta(n)$

$a = 1$ $\quad \log_b a = 0$

$b = 2$ $\quad k = 1$

$\therefore T.C. \rightarrow \Theta(n)$

(b) average case

On an average, if we use a random pivot, it should be pretty good.

∴ $\Theta(n)$

↳ T.C.

(i) worst case :

if our pivot is always the largest or smallest element

$T(n) = T(n-1) + \Theta(n)$

this is T.C $\rightarrow \Theta(n^2)$

(ii) T.C. of randomised quick sort.

let $T(n)$ = no. of oper. time to sort n elements

now, ~~lets say we~~ we

probability of choosing any element as pivot is $\frac{1}{n}$

& pivot could be first, sec ....

for 0-based index

$[0, \ldots, i, \ldots, n]$

$i-1$ | $n-i$

considering all possible

$$T(n) = \frac{1}{n}\left[T(1)\cdot T(n-1) + \Theta(n)\right] + \frac{1}{n}\left[T(2)\cdot T(n-2) + \Theta(n)\right] \ldots$$

$$\therefore T(n) = \frac{1}{n}\sum_{i=0}^{i=n-1} T(i-1) + T(n-i) + \Theta(n)$$

$$\sum_{i=1}^{i=n} T(i-1) = \sum_{i=0}^{i=n} T(n-i)$$

are symmetric

$$\therefore T(n) = \frac{2}{n}\sum_{i=1}^{i=n} T(i-1) + \Theta(n)$$

Now $\sum_{i=1}^{i=n} T(i-1) = \sum_{i=0}^{i=n-1} T(i)$

(obv.)

$$\therefore T(n) = \frac{2}{n} \sum_{i=0}^{i=n-1} T(i) + \Theta(n)$$

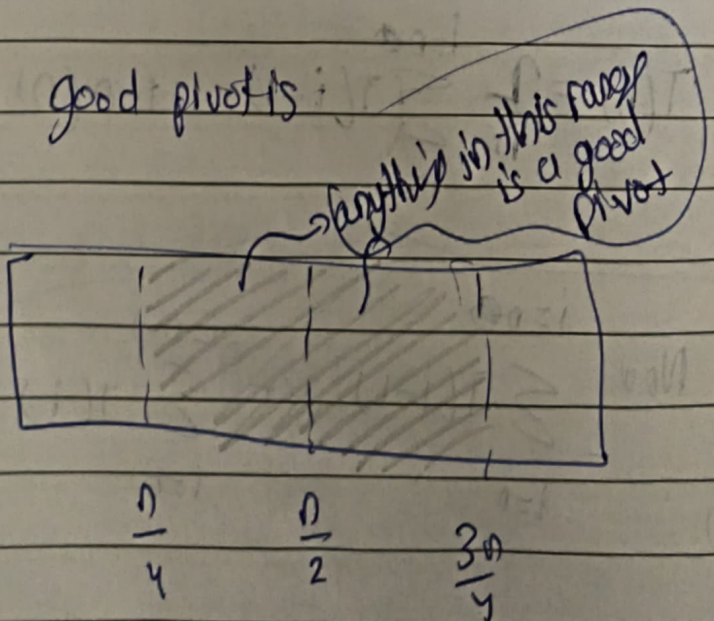
on solving recurrence relation,

we get $T(n) = \Theta(n \log_2 n)$ on an average case

Note: (* if unlucky / very low chance $\Theta(n^2)$)

worst case:

a good pivot is:



$$T(n) = \frac{2}{n} \sum_{i=0}^{i=n-1} T(i) + \frac{2}{n} \sum_{i=0}^{i=n-1} \Theta(n)$$

$\sum_{i=0}^{i=n-1} \Theta(n)$ ~ n times were going up $\Theta(n)$ & then divide by n

$$T(n) = \frac{2}{n} \sum_{i=0}^{i=n-1} T(i) + \Theta(n)$$

$$\sum_{i=0}^{i=n-1} T(i) = \frac{n}{2}[T(n) - \Theta(n)] \quad (A)$$

we try & get rid of summation;

write $T(n-1)$

$$T(n-1) = \frac{2}{n-1} \sum_{i=0}^{i=n-2} T(i) + \Theta(n)$$

$$\sum_{i=0}^{i=n-2} T(i) = \frac{(n-1)}{2}[T(n-1) - \Theta(n-1)] \quad (B)$$

So now, $\sum_{i=0}^{i=n-1} T(i) = \frac{n \cdot T(n)}{2} - \Theta(n)$ (2)

∴ using this;

$$T(n) = \frac{2}{n}\left[\frac{n}{2} \cdot T(n) - \Theta(n)\right] + \Theta(n)$$

subtract B from A;

$$\frac{n}{2} T(n) - \frac{(n-1)}{2} T(n-1) = T(n-1) + \Theta(n)$$

$$nT(n) = (n+1)\, T(n-1) + \Theta(n)$$

$$T(n) = \frac{(n+1)}{(n)} T(n-1) + \Theta(1)$$

∴ on unrolling;

$$\Rightarrow \frac{n+1}{n} \cdot \frac{n}{n-1} \cdot T(n-2) + \Theta(1) + \Theta(1) \ldots \ldots$$

n times

$$\therefore T(n) = \frac{n+1}{n} \cdot \frac{n}{n-1} \cdot \frac{n-1}{n-2} \cdot \frac{n-2}{n-3} \cdots T(n-k) + \Theta(n)$$

$$T(n) = \frac{n+1}{2} \cdot T($$

$$T(n) = \Theta\frac{n+1}{n-k} T(n-k) + c\left[1 + \frac{n+1}{n} + \frac{n-1}{n-1} + \cdots\right]$$

$$\Rightarrow T(n) = \Theta(n) + c(n+1)\left[\frac{1}{n+1} + \frac{1}{n} + \cdots \frac{1}{2}\right]$$

harmonic progression for $\log n$

$\Theta(n) \quad \Theta(\log n)$

$$\therefore T(n) = \Theta(n) + c(n+1)\log n$$

$$\therefore T(n) \longrightarrow \Theta(n \log n)$$

$$T(\cdot) \hookrightarrow \Theta(n \log n)$$